



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

گزارش پروژه معماری کامپیوتر

نگارش

ثمین صادقی پور — ۴۰۳۱۰۶۱۸۷

ریحانه حق‌گوی — ۴۰۳۱۷۰۹۴۴

سانیار رشیدی — ۴۰۳۱۰۵۹۹۶

کسری شیخی — ۴۰۳۱۱۰۵۹۶

استاد

دکتر حسین اسدی

مرداد ۱۴۰۵

فصل ۱

شرح پروژه

پروژه‌ی انتخابی گروه ما پروژه‌ی هشتم بود که در ابتدا به شرح مختصری از آن می‌پردازیم:

۱-۱ مقدمه و شرح کلی

در این پروژه، هدف اصلی بهبود کارایی یک پردازنده پایپ‌لاین ۵ مرحله‌ای MIPS ۳۲ بیتی از طریق پیاده‌سازی تکنیک هدایت داده (*DataForwarding* یا *Bypassing*) در محیط *LogisimEvolution* است. در حالت عادی، وقوع مخاطرات داده (*DataHazards*) از نوع RAW باعث توقف خط لوله (*Stall*) و کاهش چشمگیر کارایی (افت *IPC*) پردازنده می‌شود. با طراحی و اضافه کردن این واحد، نتایج محاسبه شده در مراحل جلوتر خط لوله، مستقیماً به ورودی‌های *ALU* بازگردانده می‌شوند تا از توقف‌های غیرضروری برای نوشتن و خواندن مجدد از فایل ثباتی جلوگیری شود.

۲-۱ مشخصات سیستم پایه

معماری مد نظر یک پردازنده MIPS پنج مرحله‌ای است که از دستورات متنوعی شامل دستورات نوع *R* (مانند *add, sub, and, or, slt*)، دستورات نوع *I* (مانند *lw, sw, addi, beq*) و دستورات کنترلی نوع *J* (مانند *j*) پشتیبانی می‌کند.

۳-۱ مراحل انجام و پیاده‌سازی پروژه

این پروژه طی چهار گام اصلی و برای مدیریت دو نوع مخاطره اساسی انجام می‌پذیرد:

۱. اصلاح مسیر داده در مرحله اجرا (*Execute*): قرار دادن دو مالتی‌پلکسر ۳ به ۱ در ورودی‌های *ALU*. این مالتی‌پلکسرها انتخاب می‌کنند که داده ورودی *ALU* از فایل ثابتی (*ID/EX*) تامین شود، یا از خروجی مرحله اجرا (*EX/MEM*) و یا از مرحله نوشتن (*MEM/WB*) هدایت گردد.

۲. طراحی واحد هدایت داده (*ForwardingUnit*): طراحی یک مدار ترکیبی مستقل که شماره ثابت‌های مبدأ (*Rs* و *Rt*) را از مرحله *EX* با ثابت‌های مقصد (*Rd*) در مراحل *MEM* و *WB* مقایسه می‌کند. در صورت تطابق و فعال بودن سیگنال *RegWrite*، این واحد سیگنال‌های کنترلی *ForwardA* و *ForwardB* را برای هدایت داده مناسب تولید می‌کند. همچنین اولویت‌بندی مخاطره *EX* نسبت به مخاطره *MEM* (در صورت بروز همزمان) در این منطق لحاظ می‌شود.

۳. طراحی واحد تشخیص مخاطره (*HazardDetectionUnit*): این ماژول در مرحله *Decode* قرار می‌گیرد و وظیفه تشخیص مخاطره استفاده از بارگذاری (*Load – UseHazard*) را بر عهده دارد. در صورتی که دستور مرحله *EX* یک دستور *lw* باشد و مقصد آن با مبدأ دستور فعلی در مرحله *ID* یکسان باشد، این واحد خط لوله را برای یک چرخه متوقف کرده (*Stall*)، سیگنال‌های کنترلی *PCWrite* و *IF/IDWrite* را صفر می‌کند و یک حباب (*NOP* یا *Bubble*) به مرحله *EX* می‌فرستد.

۴. ارزیابی نرم‌افزاری و محک‌زنی (*Benchmarking*): اجرای یک برنامه نمونه به زبان اسمبلی و مقایسه دقیق تعداد چرخه‌های ساعت مورد نیاز برای اجرای آن در دو حالت با و بدون واحد هدایت داده، جهت تحلیل میزان افزایش کارایی.

۴-۱ سناریوهای آزمون

درستی عملکرد پردازنده با سناریوهای مختلفی ارزیابی خواهد شد:

- هدایت داده مستقیم (*EXHazard*): ارزیابی اجرای بدون توقف دستورات متوالی که به نتیجه یکدیگر وابسته‌اند.

- تشخیص توقف ($Load - Use Hazard$): بررسی رفتار صحیح پردازنده هنگام وابستگی بلافاصله پس از دستور خواندن از حافظه.

- بخش امتیازی (اختیاری): طراحی شبکه هدایت داده اختصاصی در مرحله ID برای ارزیابی سریع‌تر شرط دستورات پرش (مانند beq)، که منجر به کاهش جریمه پرش ($Branch Penalty$) می‌شود.

در ادامه مراحل انجام شده در هر یک از بخش‌های نام برده شده را به صورت جداگانه با جزئیات بیشتر توضیح می‌دهیم:

فصل ۲

گام اول و دوم

۱-۲ مقدمه

در پردازنده‌ی پایپ‌لاین ۵ مرحله‌ای MIPS طراحی شده، در صورت وجود وابستگی داده از نوع *Read – Write After – RAW* بین دستورات متوالی، مقدار جدید یک ثبات هنوز در فایل ثباتی نوشته نشده است در حالی که دستور بعدی به همان ثبات به‌عنوان ورودی نیاز دارد. راه‌حل معمول این مشکل، متوقف کردن (*Stall*) خط لوله برای چند چرخه است تا داده در فایل ثباتی نوشته و سپس بازخوانی شود؛ اما این کار باعث افت شدید کارایی (*IPC*) پردازنده می‌شود.

در این پروژه، به‌جای توقف خط لوله، یک واحد هدایت داده (*Data Forwarding Unit*) طراحی و پیاده‌سازی شد که نتایج محاسبه‌شده در مراحل *Memory* و *Write-Back* را مستقیماً و بدون توقف به ورودی‌های *ALU* در مرحله‌ی *Execute* بازمی‌گرداند. در ادامه، دو بخش اصلی این کار یعنی اصلاح مسیر داده در مرحله‌ی *Execute* (گام اول) و طراحی منطق واحد *Forwarding* (گام دوم) به‌طور کامل شرح داده می‌شود.

۲-۲ سیگنال‌های موجود در معماری پایه

پیش از افزودن منطق هدایت داده، معماری پایه‌ی پیاده‌سازی‌شده در محیط *Logisim Evolution* از قبل شامل ثبات‌های خط لوله *ID/EX*، *EX/MEM* و *MEM/WB* بود که سیگنال‌های زیر را در اختیار می‌گذاشتند و مستقیماً برای ساخت واحد *Forwarding* مورد استفاده قرار گرفتند:

• از ثبات ID/EX : خروجی‌های EX_REG1 (شماره‌ی ثبات Rs) و EX_REG2 (شماره‌ی ثبات Rt) در مرحله‌ی $Execute$.

• از ثبات EX/MEM : خروجی‌های $MEM_DestReg$ (شماره‌ی ثبات مقصد دستوری که در مرحله‌ی $Memory$ قرار دارد)، $MEM_RegWrite$ (سیگنال کنترلی فعال بودن نوشتن) و MEM_ALU_LOW (نتیجه‌ی عددی ALU که در این ثبات نگه‌داری شده است).

• از ثبات MEM/WB : خروجی‌های $WB_DestReg$ و $WB_RegWrite$.

تنها سیگنالی که از قبل به‌صورت مجزا در دسترس نبود، مقدار نهایی نوشته‌شونده در فایل ثباتی در مرحله‌ی $Write-Back$ بود. این مقدار، خروجی زنجیره‌ای از مالتی‌پلکسرهای که بین منابع مختلف نتیجه (خروجی ALU ، داده‌ی خوانده‌شده از حافظه، نتیجه‌ی $SLTI$ ، ثبات‌های LO/HI و آدرس بازگشت در دستورات $Jump$ and $Link$) یکی را بر اساس سیگنال کنترلی $FINAL_MemtoReg$ انتخاب کرده و مستقیماً به ورودی $Data$ ماژول $RegisterFile$ می‌دهد. برای در دسترس قرار دادن این مقدار، یک $Tunnel$ جدید با برچسب $WB_WriteData$ روی همین سیم (بین خروجی آخرین مالتی‌پلکسر زنجیره و ورودی $Data$ فایل ثباتی) قرار داده شد.

۳-۲ گام اول: اصلاح مسیر داده در مرحله‌ی اجرا

پیش از افزودن هدایت داده، هر دو ورودی ALU مستقیماً از خروجی‌های $EX_RegRead1$ و $EX_RegRead2$ ثبات ID/EX (که پیش از این از طریق مالتی‌پلکسرهای موجود برای انتخاب بین ثبات، مقدار فوری ($Immediate$) و مقدار شیفت انتخاب می‌شدند) تغذیه می‌شدند. برای پیاده‌سازی هدایت داده، دو مالتی‌پلکسر ۳-به-۱ جدید با عرض داده‌ی ۳۲ بیت و عرض سلکتور ۲ بیت، با نام‌های $ForwardMux_A$ و $ForwardMux_B$ ، درست پیش از ورود این دو مقدار به منطق انتخاب موجود (شامل انتخابگر مربوط به شیفت و انتخابگر $ALUSrc$) اضافه شدند.

ورودی‌های هرکدام از این دو مالتی‌پلکسر به شرح زیر تعریف شدند:

• ورودی ۰۰: مقدار خام ثبات، یعنی $EX_RegRead1$ برای $ForwardMux_A$ و $EX_RegRead2$ برای $ForwardMux_B$ — همان مسیر پیش‌فرض که در نبود وابستگی داده مورد استفاده قرار می‌گیرد.

- ورودی ۰۱: مقدار $WB_WriteData$ ، یعنی داده‌ی نهایی نوشته‌شونده در فایل ثباتی از مرحله‌ی $Write-Back$.

- ورودی ۱۰: مقدار MEM_ALU_LOW ، یعنی خروجی عددی ALU نگه‌داشته‌شده در ثبات خط لوله‌ی EX/MEM .

- ورودی ۱۱: از آن‌جا که سیگنال‌های سلکتور ($ForwardA/ForwardB$) هرگز مقدار ۱۱ به خود نمی‌گیرند، این ورودی به یک $Constant$ صفر با عرض ۳۲ بیت متصل شد تا از باقی ماندن پین شناور و هشدارهای مربوط به آن جلوگیری شود.

سلکتور $ForwardMux_A$ به خروجی $ForwardA$ و سلکتور $ForwardMux_B$ به خروجی $ForwardB$ زیرمدار $ForwardingUnit$ متصل شد. خروجی این دو مالتی‌پلکسر جدید، جایگزین تمام مصرف‌کننده‌های قبلی $EX_RegRead\ 1$ و $EX_RegRead\ 2$ در منطق انتخاب پیش از ALU شدند؛ به این ترتیب، مقداری که وارد منطق انتخاب شیفت/فوری می‌شود، همواره از پیش هدایت‌شده است و نتیجه‌ی نهایی که به ورودی‌های a و b مازول ALU می‌رسد، همیشه شامل اثر هدایت داده خواهد بود.

۴-۲ گام دوم: طراحی زیرمدار واحد هدایت داده

برای تولید سیگنال‌های کنترلی $ForwardA$ و $ForwardB$ ، یک زیرمدار مستقل با نام $ForwardingUnit$ طراحی شد. این زیرمدار دارای شش ورودی و دو خروجی به شرح زیر است:

- ورودی‌ها: EX_Rs و EX_Rt (هرکدام ۵ بیت، شماره‌ی ثبات‌های مبدأ در مرحله‌ی $Execute$)، MEM_Rd و $MEM_RegWrite$ (شماره‌ی ثبات مقصد و سیگنال نوشتن در مرحله‌ی $Memory$)، WB_Rd و $WB_RegWrite$ (شماره‌ی ثبات مقصد و سیگنال نوشتن در مرحله‌ی $Write-Back$).

- خروجی‌ها: $ForwardA$ و $ForwardB$ (هرکدام ۲ بیت).

برای هرکدام از دو سیگنال خروجی (که منطق ساخت آن‌ها کاملاً مشابه و مستقل از یکدیگر است)، دو شرط مخاطره به‌صورت زیر با استفاده از مقایسه‌گرهای ۵ بیتی و گیت‌های منطقی AND و NOT ساخته شد:

$$EX_hazard = (MEM_Rd = EX_Rs) \wedge (MEM_Rd \neq 0) \wedge MEM_RegWrite \quad (1-2)$$

$$MEM_hazard = (WB_Rd = EX_Rs) \wedge (WB_Rd \neq 0) \wedge WB_RegWrite \wedge \overline{EX_hazard} \quad (2-2)$$

شرط $MEM_Rd \neq 0$ (و مشابه آن $WB_Rd \neq 0$) با معکوس کردن خروجی یک *Comparator* که MEM_Rd را با مقدار ثابت صفر مقایسه می‌کند به دست آمده است. وجود عبارت $\overline{EX_hazard}$ در معادله MEM_hazard تضمین می‌کند که در صورت وقوع هم‌زمان هر دو مخاطره (مثلاً در حالت نوشتن متوالی به یک ثبات توسط دو دستور پیاپی)، مخاطره‌ی مرحله‌ی *Execute* که داده‌ی جدیدتری دارد، اولویت پیدا کند.

در نهایت، دو سیگنال یک‌بیتی EX_hazard و MEM_hazard با استفاده از یک *Splitter* در حالت ترکیب (*Combine*) به یک گذرگاه ۲ بیتی تبدیل شدند؛ به گونه‌ای که بیت پرارزش (*MSB*) برابر EX_hazard و بیت کم‌ارزش (*LSB*) برابر MEM_hazard قرار گرفت. جدول درستی حاصل از این ساختار به شرح جدول زیر است:

منبع داده‌ی انتخابی	مقدار خروجی	MEM_hazard	EX_hazard
ID/EX (بدون هدایت)	۰۰	۰	۰
MEM/WB	۰۱	۱	۰
EX/MEM	۱۰	۰	۱
رخ نمی‌دهد (به دلیل شرط $\overline{EX_hazard}$)	—	۱	۱

جدول ۱-۲: جدول درستی سیگنال‌های *ForwardA* و *ForwardB*

این جدول دقیقاً منطبق بر جدول کدگذاری خواسته‌شده در سند پروژه است: مقدار ۰۰ برای حالت عادی (داده از فایل ثباتی در ID/EX)، مقدار ۱۰ برای هدایت از EX/MEM و مقدار ۰۱ برای هدایت از MEM/WB .

۵-۲ یکپارچه‌سازی نهایی

پس از تکمیل زیرمدار *ForwardingUnit*، یک نمونه از آن در مدار اصلی قرار گرفت و ورودی‌های آن با شش *Tunnel* متناظر (EX_REG1 ، EX_REG2 ، $MEM_DestReg$ ، $MEM_RegWrite$ ،

این $ForwardB$ و $ForwardA$ خروجی‌های تغذیه شدند. $(WB_RegWrite, WB_DestReg)$ زیرمدار مستقیماً به سلک‌تورهای $ForwardMux_A$ و $ForwardMux_B$ توضیح داده‌شده در بخش قبل متصل شدند. به این ترتیب، مسیر کامل هدایت داده — از تشخیص مخاطره در زیرمدار $ForwardingUnit$ تا اعمال آن روی مسیر داده‌ی ALU — بدون نیاز به هیچ توقف اضافه‌ای در خط لوله، تکمیل شد.

فصل ۳

گام سوم

۱-۳ مقدمه

همان‌طور که در بخش‌های پیشین اشاره شد، واحد *ForwardingUnit* به‌تنهایی تمام مخاطرات *RAW* را پوشش نمی‌دهد. در حالتی که دستور تولیدکننده‌ی یک مقدار از نوع *lw* باشد و بلافاصله توسط دستوری که به همان ثبات نیاز دارد دنبال شود، داده‌ی موردنیاز در لحظه‌ای که دستور دوم وارد مرحله‌ی *Execute* می‌شود هنوز از حافظه بازخوانی نشده است؛ این داده تنها در پایان مرحله‌ی *Memory* در دسترس قرار می‌گیرد. به این حالت، مخاطره‌ی *Load-Use* گفته می‌شود و تنها راه‌حل صحیح آن، متوقف کردن خط لوله برای دقیقاً یک چرخه است. در این بخش، طراحی و پیاده‌سازی واحد *Hazard Detection* که مسئول تشخیص این وضعیت و اعمال توقف کنترل‌شده است، شرح داده می‌شود.

۲-۳ سیگنال‌های موجود مورد استفاده

برای تشخیص مخاطره‌ی *Load-Use*، نیازی به افزودن سیگنال جدید به ثبات‌های خط لوله نبود؛ تمام اطلاعات لازم پیش از این در دسترس بود:

- از ثبات *ID/EX*: خروجی‌های *EX_MemtoReg* و *EX_RegWrite* (سیگنال‌های کنترلی دستوری که هم‌اکنون در مرحله‌ی *Execute* قرار دارد) و همچنین *EX_REG2*. از آن‌جا که در معماری این پروژه دستور *lw* از نوع *I* است، ثبات مقصد آن همواره در فیلد *Rt* قرار دارد؛ بنابراین برای دستوری که واقعاً یک *lw* باشد، مقدار *EX_REG2* دقیقاً همان ثبات مقصد آن است و نیازی

به استفاده از خروجی جداگانه‌ی پس از مالتی‌پلکسر $RegDst$ نبود.

- از مرحله‌ی $Decode$ (پیش از ثبات ID/EX): سیگنال‌های $REG1$ و $REG2$ که به ترتیب شماره‌ی ثبات‌های Rt و Rs دستوری هستند که هم‌اکنون در حال کدگشایی است. این دو سیگنال در زیرمدار جدید با نام‌های ID_REG1 و ID_REG2 مورد استفاده قرار گرفتند.

۳-۳ طراحی زیرمدار $Hazard_Detection$

زیرمداری مستقل با نام $Hazard_Detection$ طراحی شد که پنج ورودی $EX_MemtoReg$ ، EX_REG2 ، $EX_RegWrite$ ، ID_REG1 و ID_REG2 را دریافت کرده و یک خروجی یک‌بیتی به نام $Stall$ تولید می‌کند. منطق داخلی این زیرمدار از سه بخش تشکیل شده است:

نخست، سیگنال Is_Load که مشخص می‌کند آیا دستور موجود در مرحله‌ی $Execute$ واقعاً یک بارگذاری از حافظه است، از طریق یک گیت AND ساده بین $EX_MemtoReg$ و $EX_RegWrite$ ساخته شد:

$$Is_Load = EX_MemtoReg \wedge EX_RegWrite \quad (۱-۳)$$

سپس، سیگنال $Match$ که برابری ثبات مقصد دستور در حال اجرا با یکی از ثبات‌های مبدأ دستور در حال کدگشایی را بررسی می‌کند، با استفاده از دو $Comparator$ عرض ۵ بیت و یک گیت OR ساخته شد:

$$Match = (EX_REG2 = ID_REG1) \vee (EX_REG2 = ID_REG2) \quad (۲-۳)$$

در نهایت، با استفاده از یک $Comparator$ سوم که EX_REG2 را با مقدار ثابت صفر مقایسه می‌کند و یک گیت NOT روی خروجی آن، شرط $Dest_Zero$ برای حذف حالت نوشتن در ثبات $zero$ ساخته شد. خروجی نهایی $Stall$ از ترکیب این سه سیگنال توسط یک گیت AND سه‌ورودی به دست می‌آید:

$$Stall = Is_Load \wedge Match \wedge \overline{Dest_Zero} \quad (۳-۳)$$

جدول درستی این واحد به شرح زیر است:

<i>Stall</i>	$EX_REG \neq 0$	<i>Match</i>	<i>Is Load</i>
0	×	×	0
0	×	0	1
0	0	1	1
1	1	1	1

جدول ۳-۱: جدول درستی سیگنال *Stall* در واحد *Hazard_Detection*

۴-۳ یکپارچه‌سازی نهایی

پس از تکمیل زیرمدار، یک نمونه از آن در مدار اصلی قرار گرفت و ورودی‌های آن با پنج *Tunnel* متناظر تغذیه شدند. خروجی *Stall* این نمونه، از طریق یک *Tunnel* با همین نام، به یک گیت *NOT* متصل شد که سیگنال *NOT_STALL* را تولید می‌کند. این سیگنال در سه نقطه‌ی مختلف مدار اصلی مورد استفاده قرار گرفت:

- ورودی *enable* ثبات‌های خط لوله *PC* و *IF/ID*، که با فعال شدن *Stall* متوقف می‌شوند و در نتیجه نه واکنشی دستور جدید و نه پیشروی دستور موجود در مرحله‌ی *Decode* انجام نمی‌شود.
- دو گیت *AND* که سیگنال‌های *RegWrite* و *MemWrite* خروجی واحد کنترل را — پیش از ورود به ثبات *ID/EX* — با *NOT_STALL* ترکیب می‌کنند و خروجی‌های *ID_RegWrite* و *ID_MemWrite* را تولید می‌کنند. به این ترتیب، در چرخه‌ای که مخاطره‌ی *Load-Use* تشخیص داده می‌شود، به جای سیگنال‌های کنترلی واقعی دستور موجود در *Decode*، یک حباب (*NOP*) با $RegWrite = 0$ و $MemWrite = 0$ وارد ثبات *ID/EX* می‌شود؛ سایر سیگنال‌های کنترلی (مانند *ALUOp* یا *RegDst*) از آن جا که بدون فعال بودن *RegWrite* یا *MemWrite* اثری بر وضعیت پردازنده ندارند، بدون تغییر باقی می‌مانند.

با این طراحی، در سناریوی دوم پروژه — یعنی قرارگیری دستور *lw* در مرحله‌ی *Execute* و دستور وابسته به آن در مرحله‌ی *Decode* — پردازنده دقیقاً برای یک چرخه متوقف می‌شود: *PC* و *IF/ID* ثابت می‌مانند، یک حباب وارد *ID/EX* می‌شود، و در چرخه‌ی بعد، دستور وابسته به‌طور طبیعی وارد *Execute* شده و با استفاده از منطق *ForwardingUnit* (که در این لحظه دستور *lw* را در مرحله‌ی *Write-Back* می‌بیند) مقدار صحیح از طریق *ForwardMux* با کد ۰۱ به ورودی *ALU* هدایت می‌شود؛ دقیقاً همان رفتاری که در سناریوهای پروژه توصیف شده است.

فصل ۴

سناریوهای آزمون و ارزیابی گام چهارم

در این فصل، نتایج دو سناریوی اجباری پروژه و ارزیابی نرم‌افزاری گام چهارم ارائه می‌شود. هدف سناریوی نخست بررسی هدایت مستقیم داده از ثبات خط لوله‌ی EX/MEM و هدف سناریوی دوم بررسی تشخیص مخاطره‌ی $Load-Use$ ، ایجاد دقیقاً یک حباب و سپس هدایت داده از MEM/WB است. در انتها نیز برنامه‌ی فیبوناچی روی پردازنده‌ی مجهز به واحد هدایت داده و پردازنده‌ی پایه‌ی بدون هدایت اجرا و تعداد چرخه‌ها مقایسه می‌شود.

۴-۱ سناریوی اول: مخاطره‌ی EX و هدایت مستقیم

در این سناریو دو دستور وابسته‌ی زیر به صورت متوالی اجرا شدند:

```
add $t0, $t1, $t2
```

```
sub $t3, $t0, $t4
```

مقادیر اولیه برابر با $t_1 = 10$ ، $t_2 = 3$ و $t_4 = 4$ در نظر گرفته شد. در نتیجه، دستور اول مقدار ۱۳ را در ثبات t_0 تولید می‌کند. هنگام اجرای دستور دوم، این مقدار هنوز در فایل ثبات نوشته نشده است؛ بنابراین واحد هدایت داده تطابق ثبات مقصد دستور اول با ثبات مبدأ دستور دوم را تشخیص داده و سیگنال $ForwardA = 10$ را تولید می‌کند. به این ترتیب مقدار جدید مستقیماً از مسیر EX/MEM به ورودی اول واحد محاسبه و منطق هدایت می‌شود.

نتیجه‌ی نهایی برابر با $t_0 = 13$ و $t_3 = 9$ بود. همچنین بین پایان دو دستور هیچ چرخه‌ی خالی

مشاهده نشد؛ بنابراین هدایت مستقیم بدون توقف خط لوله به درستی انجام شده است. خروجی آزمون در شکل ۴-۱ نمایش داده شده است.

```
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /Users/kasra/logisim_evolution_workspace/pipeline_v3.circ.tmp/main/verilog/circuit/main.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /Users/kasra/logisim_evolution_workspace/pipeline_v3.circ.tmp/main/verilog/toplevel/logisimTopLevelShell.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /Users/kasra/logisim_evolution_workspace/pipeline_v3.circ.tmp/main/scripts/vivadoCreateProject.tcl
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /Users/kasra/logisim_evolution_workspace/pipeline_v3.circ.tmp/main/xdc/vivadoConstraints.xdc
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /Users/kasra/logisim_evolution_workspace/pipeline_v3.circ.tmp/main/scripts/vivadoGenerateBitStream.tcl
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /Users/kasra/logisim_evolution_workspace/pipeline_v3.circ.tmp/main/scripts/vivadoLoadBitStream.tcl

tests/scenarios/tb_scenario1_ex_hazard.v /Users/kasra/logisim_evolution_workspace/pipeline_v3.circ.tmp
commit=0 cycle=1 instruction=0x2009000a
commit=1 cycle=2 instruction=0x200a0003
commit=2 cycle=3 instruction=0x200c0004
commit=3 cycle=4 instruction=0x20100065
commit=4 cycle=5 instruction=0x20110066
commit=5 cycle=6 instruction=0x20120067
commit=6 cycle=7 instruction=0x012a4020
commit=7 cycle=8 instruction=0x010c5822
PASS: EX/MEM value reached ALU input A with no stall
ACCEPTED_SCENARIO_1_EX_HAZARD

ca-practical-4042/Project
>> |
```

شکل ۴-۱: خروجی سناریوی اول و پذیرش هدایت مستقیم از EX/MEM بدون توقف

۴-۲ سناریوی دوم: مخاطره‌ی بارگذاری-استفاده

در سناریوی دوم، دستورات زیر به صورت متوالی اجرا شدند:

```
lw    $t0, 0($t1)
add   $t2, $t0, $t3
```

برای اجرای آزمون، خانه‌ی حافظه‌ی داده با آدرس ۱۶ برابر با ۳۷ قرار داده شد و مقادیر اولیه‌ی $t_1 = ۱۶$ و $t_3 = ۵$ انتخاب شدند. هنگامی که دستور lw در مرحله‌ی EX و دستور add در مرحله‌ی ID قرار دارد، داده‌ی حافظه هنوز آماده نیست و هدایت مستقیم به‌تنهایی کافی نخواهد بود. واحد تشخیص مخاطره این وابستگی را شناسایی می‌کند، به مدت دقیقاً یک چرخه از به‌روزرسانی PC و ثبات IF/ID جلوگیری کرده و یک حباب در ID/EX قرار می‌دهد.

در چرخه‌ی بعد، مقدار بارگذاری شده از مسیر MEM/WB با انتخاب $ForwardA = ۰۱$ به ورودی

واحد محاسبه و منطق می‌رسد. نتیجه‌ی نهایی آزمون برابر با $t_0 = 37$ و $t_2 = 42$ بود. شمارش چرخه‌ها نیز وجود دقیقاً یک توقف را تأیید کرد. خروجی این آزمون در شکل ۴-۲ آمده است.

```

v3.circ.tmp/main/verilog/circuit/ControlUnit.v
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /Users/kasra/logisim_evolution_workspace/pipeline_
v3.circ.tmp/main/verilog/circuit/main.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /Users/kasra/logisim_evolution_workspace/pipeline_
v3.circ.tmp/main/verilog/toplevel/logisimTopLevelShell.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /Users/kasra/logisim_evolution_workspace/pipeli
ne_v3.circ.tmp/main/scripts/vivadoCreateProject.tcl
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /Users/kasra/logisim_evolution_workspace/pipeli
ne_v3.circ.tmp/main/xdc/vivadoConstraints.xdc
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /Users/kasra/logisim_evolution_workspace/pipeli
ne_v3.circ.tmp/main/scripts/vivadoGenerateBitStream.tcl
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /Users/kasra/logisim_evolution_workspace/pipeli
ne_v3.circ.tmp/main/scripts/vivadoLoadBitStream.tcl

tests/scenarios/tb_scenario2_load_use.v /Users/kasra/logisim_evolution_workspace/pipeline_v3.circ.tmp
commit=0 cycle=1 instruction=0x20090010
commit=1 cycle=2 instruction=0x2008000a
commit=2 cycle=3 instruction=0x200b0005
commit=3 cycle=4 instruction=0x2010007b
commit=4 cycle=6 instruction=0x8d280000
PASS: exactly one load-use stall observed
commit=5 cycle=7 instruction=0x010b5020
PASS: MEM/WB load value was forwarded after one bubble
ACCEPTED_SCENARIO_2_LOAD_USE

ca-practical-4042/Project

```

شکل ۴-۲: خروجی سناریوی دوم، مشاهده‌ی یک حباب و هدایت مقدار بارگذاری شده از MEM/WB

۳-۴ گام چهارم: ارزیابی نرم‌افزاری و محک‌زنی

برای ارزیابی اثر واحد هدایت داده، برنامه‌ی حلقه‌ای محاسبه‌ی فیبوناچی به زبان اسمبلی نوشته و به کد ماشین تبدیل شد. مقدارهای آغازین $F(0) = 0$ و $F(1) = 1$ بودند و شمارنده‌ی حلقه برابر با ۸ قرار گرفت. بخش اصلی برنامه به‌صورت زیر است:

```

loop:
    add    $v0, $t0, $t1
    add    $t0, $t1, $zero
    add    $t1, $v0, $zero
    addi   $a0, $a0, -1
    bnez   $a0, loop
    nop

```

done:

```
j      done
nop
```

دو نسخه‌ی معادل از برنامه تهیه شد. در نسخه‌ی پردازنده‌ی مجهز به هدایت داده، تمام *NOP* های مربوط به مخاطرات داده حذف شدند و تنها *NOP* های مربوط به تأخیر کنترل پس از پرش باقی ماندند. در نسخه‌ی پردازنده‌ی بدون هدایت، *NOP* ها به صورت دستی بین دستورات وابسته قرار گرفتند. سه *NOP* پس از مقداردهی اولیه، سه *NOP* میان انتقال مقادیر فیبوناچی و چهار *NOP* پیش از دستور پرش استفاده شد. با توجه به اجرای هشت باره‌ی حلقه، تعداد پویای *NOP* های داده در نسخه‌ی پایه برابر با ۵۹ است.

شرط پایان اندازه‌گیری، مشاهده‌ی هم‌زمان $a_0 = 0$ ، $t_0 = 21$ ، $t_1 = 34$ و $v_0 = 34$ بود. بنابراین هر دو پردازنده مقدار نهایی یکسان و صحیح ۳۴ را تولید کردند و تنها تعداد چرخه‌های لازم متفاوت بود.

جدول ۴-۱: مقایسه‌ی اجرای برنامه‌ی فیبوناچی

پردازنده	تعداد چرخه	<i>NOP</i> داده‌ی پویا	خروجی نهایی
با واحد هدایت داده	۵۴	۰	۳۴
بدون واحد هدایت داده	۱۰۹	۵۹	۳۴

بر اساس نتایج جدول ۴-۱، تعداد چرخه‌های صرفه‌جویی شده و شتاب به صورت زیر به دست می‌آیند:

$$\text{Cycles Saved} = 109 - 54 = 55$$

$$\text{Cycle Reduction} = \frac{55}{109} \times 100 \simeq 50.5\%$$

$$\text{Speedup} = \frac{109}{54} \simeq 2.02$$

در نتیجه، واحد هدایت داده تعداد چرخه‌ها را از ۱۰۹ به ۵۴ کاهش داده است؛ یعنی کاهش ۵۰/۵ درصدی در تعداد چرخه‌ها و شتاب تقریباً ۲/۰۲ برابر. خروجی کامل اجرای هر دو نسخه و مقایسه‌ی چرخه‌ها در شکل ۴-۳ نشان داده شده است.

۴-۴ جمع‌بندی

هر دو سناریوی اجباری با موفقیت اجرا شدند. سناریوی اول نشان داد که وابستگی *EX* بدون توقف و از مسیر *EX/MEM* برطرف می‌شود. سناریوی دوم نیز ایجاد دقیقاً یک حباب برای مخاطره‌ی *Load-Use* و هدایت داده از *MEM/WB* را تأیید کرد. در ارزیابی گام چهارم، هر دو پردازنده پاسخ صحیح فیبوناچی

```
Project -- zsh -- 166x43

[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "ID_EX" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "ID_EX" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "ID_EX" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "ID_EX" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "ID_EX" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "ID_EX" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "ID_EX" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "ID_EX" has a gated clock connection!
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /root/logisim_evolution_workspace/pipeline.circ.tmp/main/verilog/circuit/ID_EX.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /root/logisim_evolution_workspace/pipeline.circ.tmp/main/verilog/gates/OR_GATE_5_INPUTS.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /root/logisim_evolution_workspace/pipeline.circ.tmp/main/verilog/circuit/ControlUnit.v
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] WARN com.cburch.logisim.fpga.gui.Reporter - Component "Register" in circuit "main" has a gated clock connection!
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /root/logisim_evolution_workspace/pipeline.circ.tmp/main/verilog/circuit/main.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /root/logisim_evolution_workspace/pipeline.circ.tmp/main/verilog/topLevel/LogisimTopLevelShell.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /root/logisim_evolution_workspace/pipeline.circ.tmp/main/scripts/vivadoCreateProject.tcl
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /root/logisim_evolution_workspace/pipeline.circ.tmp/main/xdc/vivadoConstraints.xdc
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /root/logisim_evolution_workspace/pipeline.circ.tmp/main/scripts/vivadoGenerateBitStream.tcl
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /root/logisim_evolution_workspace/pipeline.circ.tmp/main/scripts/vivadoLoadBitStream.tcl

Project/tests/benchmark/tb_fibonacci_no_forwarding.v /root/logisim_evolution_workspace/pipeline.circ.tmp
BENCHMARK_RESULT mode=no_forwarding fib_term_10=34 static_data_nops=10 total_cycles=109 inst_done_pulses=106
ACCEPTED_FIBONACCI_NO_FORWARDING

FIBONACCI CYCLE COMPARISON
-----
With forwarding      54 cycles |#####|
Without forwarding   109 cycles |#####|
-----
Cycles saved:      55
Cycle reduction:   50.5%
Speedup:          2.02x
Final Fibonacci value: 34

Cycle summary saved to: /Users/kasra/Desktop/university/term 4/ca-practical-4042/Project/build/fibonacci_cycle_summary.txt

ca-practical-4042/Project
))
```

شکل ۴-۳: خروجی اجرای فیبوناچی و مقایسه‌ی ۵۴ چرخه با هدایت داده و ۱۰۹ چرخه بدون هدایت داده

را تولید کردند، اما پردازنده‌ی مجهز به واحد هدایت داده با حذف توقف‌های غیرضروری، اجرای برنامه را با شتاب حدود ۲/۰۲ برابر به پایان رساند.

فصل ۵

بخش امتیازی

۵-۱ هدایت داده در مرحله‌ی Decode برای مقایسه‌گر پرش

۵-۱-۱ ضرورت طراحی

در معماری پایه، برای کاهش جریمه‌ی چرخه‌ای دستورات پرش، مقایسه‌ی شرط دستور *beq* در مرحله‌ی *Decode* انجام می‌شود؛ به این صورت که شماره‌ی ثبات‌های عملوند از دستور در حال رمزگشایی استخراج و مقادیر متناظرشان به صورت ترکیبی (*Combinational*) از *RegisterFile* خوانده می‌شوند تا نتیجه‌ی مقایسه بلافاصله در همان چرخه در انتخاب مقدار بعدی *PC* اثر بگذارد. این طراحی که جریمه‌ی پرش را به کمترین مقدار ممکن می‌رساند، مستلزم آن است که مقادیر خوانده‌شده از *RegisterFile* همواره معتبر و به‌روز باشند.

اما در همان چرخه‌ای که دستور *beq* در مرحله‌ی *Decode* قرار دارد، ممکن است تا سه دستور قبلی هنوز مقدار جدید یکی از ثبات‌های *Rs* یا *Rt* را در فایل ثباتی ننوشته باشند (چون نوشتن تنها در مرحله‌ی *Write-Back* انجام می‌شود). در چنین حالتی، خواندن مستقیم از *RegisterFile* مقداری قدیمی و نامعتبر بازمی‌گرداند. از آنجا که مرحله‌ی *Decode* زودتر از مرحله‌ی *Execute* در خط لوله قرار دارد، این مسئله حتی حساس‌تر از وابستگی‌های داده‌ی معمول در ورودی‌های *ALU* است، چرا که فاصله‌ی زمانی بین دستور تولیدکننده و لحظه‌ی مصرف مقدار در *Decode* می‌تواند شامل هر سه مرحله‌ی *Execute*، *Memory* و *Write-Back* باشد. برای حل این مسئله بدون نیاز به توقف (*Stall*) خط لوله، یک شبکه‌ی هدایت داده‌ی اختصاصی برای هر دو عملوند دستور *beq* طراحی و به مسیر داده‌ی مرحله‌ی *Decode* اضافه شد.

۵-۱-۲ طراحی زیرمدار *BranchForwardingUnit*

زیرمدار *BranchForwardingUnit* با هفت ورودی زیر طراحی شد:

- ID_Rt و ID_Rs : شماره‌ی ثبات‌های عملوند دستور *beq* در مرحله‌ی *Decode* (هرکدام ۵ بیت).
- EX_Rd ، $EX_RegWrite$: شماره‌ی ثبات مقصد و سیگنال نوشتن دستور در مرحله‌ی *Execute*.

- MEM_Rd ، $MEM_RegWrite$: مربوط به دستور در مرحله‌ی *Memory*.

- WB_Rd ، $WB_RegWrite$: مربوط به دستور در مرحله‌ی *Write-Back*.

و دو خروجی مستقل *BranchForwardA* و *BranchForwardB* (هرکدام ۲ بیت) تولید می‌کند که به ترتیب مسیر هدایت داده برای Rt و Rs را کنترل می‌کنند. از آن‌جا که دستور تولیدکننده‌ی مقدار می‌تواند در سه مرحله‌ی مختلف قرار داشته باشد، برای هرکدام از دو مسیر، سه سطح اولویت با معادلات زیر تعریف شد (معادلات مسیر Rt کاملاً مشابه است و تنها ID_Rt با ID_Rs جایگزین می‌شود):

$$EX_hazard_{Rs} = (EX_Rd = ID_Rs) \wedge (EX_Rd \neq \bullet) \wedge EX_RegWrite \quad (۱-۵)$$

$$MEM_hazard_{Rs} = (MEM_Rd = ID_Rs) \wedge (MEM_Rd \neq \bullet) \wedge MEM_RegWrite \\ \wedge \overline{EX_hazard_{Rs}} \quad (۲-۵)$$

$$WB_hazard_{Rs} = (WB_Rd = ID_Rs) \wedge (WB_Rd \neq \bullet) \wedge WB_RegWrite \\ \wedge \overline{EX_hazard_{Rs}} \wedge \overline{MEM_hazard_{Rs}} \quad (۳-۵)$$

هرکدام از این شرایط با استفاده از دو *Comparator* عرض ۵ بیت (یکی برای بررسی نابرابری ثبات مقصد با ثبات صفر و دیگری برای بررسی برابری آن با ID_Rt یا ID_Rs) و گیت‌های *NOT/AND* ساخته شدند. از آن‌جا که شرط‌های $EX_Rd \neq \bullet$ ، $MEM_Rd \neq \bullet$ و $WB_Rd \neq \bullet$ مستقل از انتخاب Rt یا Rs هستند، خروجی این سه *Comparator* در ساخت هر دو مسیر *BranchForwardA* و *BranchForwardB* به طور مشترک استفاده شد و نیازی به تکرار آن‌ها نبود.

برای تبدیل سه سیگنال یک‌بیتی هر مسیر به یک خروجی ۲ بیتی با رعایت اولویت، از یک زنجیره‌ی سه مالتی‌پلکسر تودرتو استفاده شد: نخست بین مقدار ثابت ۰۰ و ۱۱ با سلکتور WB_hazard ، سپس

بین نتیجه‌ی مرحله‌ی قبل و مقدار ثابت ۱۰ با سلکتور MEM_hazard ، و در نهایت بین نتیجه‌ی مرحله‌ی قبل و مقدار ثابت ۰۱ با سلکتور EX_hazard . جدول درستی حاصل (که برای هر دو مسیر $BranchForwardA$ و $BranchForwardB$ به‌طور یکسان صدق می‌کند) به شرح زیر است:

منبع داده	مقدار خروجی	WB_hazard	MEM_hazard	EX_hazard
مقدار خام از $RegisterFile$ (بدون هدایت)	۰۰	۰	۰	۰
ALU_LOW (مرحله‌ی $Execute$)	۰۱	×	×	۱
MEM_ALU_LOW (مرحله‌ی $Memory$)	۱۰	×	۱	۰
$WB_WriteData$ (مرحله‌ی $Write-Back$)	۱۱	۱	۰	۰

جدول ۵-۱: جدول درستی سیگنال‌های $BranchForwardA$ و $BranchForwardB$

۳-۱-۵ یکپارچه‌سازی با مسیر داده‌ی مرحله‌ی Decode

برای اعمال نتیجه‌ی این واحد، دو مالتی‌پلکسر ۴-به-۱ با عرض داده‌ی ۳۲ بیت و عرض سلکتور ۲ بیت در مسیر ورودی مقایسه‌گر پرش قرار گرفتند:

• $BranchForwardMux_A$: با سلکتور $BranchForwardA$ ، بین مقدار خام $RegRead1$ (ورودی ۰۰)، ALU_LOW (ورودی ۰۱)، MEM_ALU_LOW (ورودی ۱۰) و $WB_WriteData$ (ورودی ۱۱) انتخاب می‌کند.

• $BranchForwardMux_B$: با سلکتور $BranchForwardB$ و ساختاری کاملاً مشابه، اما با $RegRead2$ به‌عنوان ورودی ۰۰.

از آن‌جا که سیگنال‌های سلکتور در این بخش هر چهار مقدار ممکن را به خود می‌گیرند، برخلاف مالتی‌پلکسرهای هدایت داده در بخش اصلی پروژه (که یک ورودی خالی و متصل به ثابت صفر داشتند)، در این‌جا هر چهار ورودی هر دو مالتی‌پلکسر به‌طور کامل استفاده شده‌اند.

مقایسه‌گر شرط پرش به‌گونه‌ای طراحی شد که خروجی $BranchForwardMux_A$ (مقدار هدایت‌شده‌ی Rs) را مستقیماً با خروجی $BranchForwardMux_B$ (مقدار هدایت‌شده‌ی Rt) مقایسه می‌کند و خروجی $A = B$ آن مستقیماً سیگنال $SHOULD_BRANCH$ را می‌سازد. این ساختار دقیقاً معادل تعریف استاندارد دستور beq است: پرش در صورت برابری مقادیر (هدایت‌شده‌ی) دو ثبات Rs و Rt ، مستقل از این‌که مقدار هرکدام از این دو ثبات از فایل ثباتی خوانده شده یا به‌تازگی توسط یکی از سه دستور قبلی محاسبه شده باشد.

۴-۱-۵ برهم‌کنش با واحد تشخیص مخاطره

از آن‌جا که *Hazard_Detection* موجود تنها برای مخاطره‌ی *Load-Use* طراحی شده است، بررسی شد که آیا این واحد نیاز به تغییر دارد یا خیر. تنها حالت خاصی که شبکه‌ی هدایت داده به‌تنهایی نمی‌تواند آن را پوشش دهد، زمانی است که دستور تولیدکننده‌ی *Rs* یا *Rt* یک دستور *lw* باشد و بلافاصله پیش از دستور *beq* قرار گیرد؛ در این حالت داده‌ی حافظه هنوز در مرحله‌ی *Execute* آماده نیست. این حالت همان مخاطره‌ی استاندارد *Load-Use* است و به‌طور خودکار توسط واحد *Hazard_Detection* موجود (که یک چرخه توقف ایجاد می‌کند) پوشش داده می‌شود، بدون نیاز به هیچ تغییری در آن.